

11-768 HW2: Evaluating a Data-Visualization Agent

Your Name

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

The baseline validator struggles significantly on the seed runs, yielding poor macro-MCC (0.0268). The major disagreements concentrate in `extttexecution_failure` where it hallucinates false positives due to rigid parsing of LLM prefixes, and in `extttwrong_chart` where it hallucinates visual critiques for non-existent figures because it failed to short-circuit upon a missing file.

1.2 Validator Error Type 1

The first recurring error type is `extttexecution_failure` False Positives. - Runs affected: 023, 083 - Evidence: In both runs, the trajectory completed normally and saved a valid `figure.png`. Ground truth specifies no execution failure. However, the baseline validator predicted an `extttexecution_failure`. The LLM actually reasoned "Yes, the agent successfully completed the task..." and the validator script parsed the "Yes" prefix and incorrectly mapped it to a failure flag.

1.3 Validator Error Type 2

The second recurring error type is `extttexecution_failure` False Negatives (Misclassification into `extttwrong_chart`). - Runs affected: 096, 208 - Evidence: In these runs, the agent terminated early or saved to the wrong filename, resulting in no `figure.png` being produced. Ground truth correctly flags these as `extttexecution_failure` and short-circuits evaluation. The baseline validator missed this, proceeded to chart evaluation, and generated highly specific hallucinations about the visual chart design for an image that does not exist.

1.4 Proposed Validator Changes

1. For False Positives: Implement a programmatic file check. Before prompting the LLM, verify that `exttttrun.figure` exists and decodes properly via PIL. 2. For False Negatives: Implement deterministic terminal short-circuiting. If no valid figure exists, immediately return an `extttexecution_failure` Error and do not query downstream visual/chart checks.

2 Part 2: Improve Your Validator

2.1 Implemented Changes

1. Deterministic figure verification: I verify file existence and use `extttPIL.Image.verify()` to ensure the image is valid. If it fails, I immediately return `extttexecution_failure`. 2. Terminal Short-circuiting: The validation function now checks the deterministic execution flag first. If an error is found, it short-circuits.

2.2 Seed-Set Performance vs. Baseline

Baseline: execution_failure: MCC=-0.0022 wrong_data: MCC=0.0000 wrong_chart: MCC=0.1096
hard_to_read: MCC=0.0000 macro_mcc: 0.0268

Modified Validator: execution_failure: MCC=1.0000 wrong_data: MCC=-0.1445 wrong_chart: MCC=-0.3798 hard_to_read: MCC=-0.2424 macro_mcc: 0.0583

2.3 Qualitative Analysis of Targeted Error Types

The changes perfectly mitigated both execution failure errors. The programmatic gate eradicated false positives and negatives, raising the MCC for extttexecution_failure to 1.0000. It also prevented hallucinations on missing figures. However, regressions appeared in extttwrong_chart and extttwrong_data MCC because the VLM is extremely rigid and struggles with fine-grained spatial constraints.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

1. Tasks with Dense or High-Precision Data: VLMs lack the precision to count overlapping thousands of points or exact complex mathematical curves. 2. Tasks with Highly Specific Formatting Constraints: Broad natural language prompts gloss over strict, esoteric formatting constraints (like exact pixel sizes or marker subsets).

3.2 Task Design

1. dense-scatter-overlap (Class 1): 10,000 overlapping points in two classes. Tests if validator can count dense points. 2. high-frequency-sine (Class 1): High frequency waves causing aliasing. Tests mathematical tracing. 3. strict-formatting-colors (Class 2): 3-bar chart requiring exact hex color codes. Tests exact formatting adherence. 4. exact-marker-grid (Class 2): 2x2 grid requiring different specific markers per panel. Tests fine-grained spatial attention. 5. tiny-font-clutter (Class 2): Pie chart with 4-point font labels on dark background. Tests extttthard_to_read contrast/legibility tracking.

3.3 Agent Runs and Labeling

I generated 15 runs across the 3 provided agents. I labeled them manually by meticulously checking code outputs against constraints. Ambiguities were resolved by heavily punishing minor formatting violations per the strict constraints authored.

3.4 Validator Performance on New Tasks

execution_failure: MCC=1.0000 wrong_data: MCC=0.0000 wrong_chart: MCC=-0.1581 hard_to_read: MCC=0.2000 macro_mcc: 0.2605

The validator maintained perfect execution checking but failed massively on extttwrong_chart because the VLM failed to notice exact hex colors or distinct markers in the grid.

3.5 Harbor Packaging

Every authored task scores 1.0 against `-a oracle`.

I implemented two mutants for `strict-formatting-colors`: 1. `wrong-data` (Reward: 0.0): This mutant plotted incorrect bar values. It was caught by the deterministic assertion `assert data.get("values") == [10, 20, 15]` inside `test_plotted_values` because the JSON output carried the wrong data. 2. `wrong-colors` (Reward: 1.0): This mutant plotted the correct data but used generic red/green/blue instead of the required hex colors. The deterministic verifier accepted it because it strictly checks `plotted_values.json` and completely ignores visual content in `figure.png`.

A deterministic Harbor verifier could easily decide `extttexecution_failure` and `extttwrong_data` (by reading the sidecar JSON). However, it struggles heavily with `extttwrong_chart` or `extttthard_to_read` because visually asserting font sizes, colors, and overlapping text deterministically via pixels requires heavy CV heuristics. Therefore, a per-task verifier cannot serve as a generalized grading mechanism across arbitrary visualization tasks without massive manual engineering per task.

4 Part 4: Iterate on Your Design

4.1 Additional Improvement Motivated by Part 3

Motivated by the failure of the VLM to catch strict constraints in Part 3 (such as exact hex colors or distinct subplot markers) and its blindness to dense overlaps making charts unreadable, I made two granular changes: 1. **Granular `extttwrong_chart`**: I changed the single broad `extttwrong_chart` prompt to a systematic extraction prompt ("Step 1: Extract all constraints... Step 2: Verify each constraint..."). I expected this "Chain of Thought" extraction to reduce hallucinations. 2. **Granular `extttthard_to_read`**: I explicitly listed four common readability issues (illegible overlaps, tiny fonts, poor contrast, extreme dense clutter) inside the prompt. I expected this to help the VLM identify the specific edge-cases presented in our new tasks (e.g., `exttttiny-font-clutter`).

4.2 Final Evaluation on Seed and New Tasks

Seed Runs (Before Change): `execution_failure`: MCC=1.0000 `wrong_data`: MCC=-0.1445 `wrong_chart`: MCC=-0.3798 `hard_to_read`: MCC=-0.2424 `macro_mcc`: 0.0583

Seed Runs (After Change): `execution_failure`: MCC=1.0000 `wrong_data`: MCC=-0.2363 `wrong_chart`: MCC=0.0000 `hard_to_read`: MCC=0.0454 `macro_mcc`: 0.2023

Self-Authored Runs (Before Change): `execution_failure`: MCC=1.0000 `wrong_data`: MCC=0.0000 `wrong_chart`: MCC=-0.1581 `hard_to_read`: MCC=0.2000 `macro_mcc`: 0.2605

Self-Authored Runs (After Change): `execution_failure`: MCC=1.0000 `wrong_data`: MCC=0.0000 `wrong_chart`: MCC=0.0000 `hard_to_read`: MCC=0.0000 `macro_mcc`: 0.2500

4.3 Qualitative Analysis of Self-Authored Tasks

The granular constraint extraction successfully eradicated False Positives on `extttwrong_chart` for the self-authored tasks. By forcing the model to systematically extract constraints, it stopped hallucinating non-existent formatting rules, bringing the False Positives for `extttwrong_chart` from 7 down to 0.

For `extttthard_to_read`, the MCC actually dropped from 0.2000 to 0.0000 on the custom tasks, despite jumping into positive territory (0.0454) on the massive Seed dataset. This indicates that

while explicitly listing common readability issues like "tiny fonts" and "poor contrast" helped the model detect readability flaws on general charts (Seed dataset), it caused the VLM to become hyper-sensitive and over-penalize our specific self-authored charts, resulting in 10 False Positives on our small custom test set.

4.4 Further Changes

We implemented several further iterations to push the validator's performance and structural safety beyond Part 4.

Iteration 1: Granular Readability Checklist

I implemented a second improvement targeting `hard_to_read`. The baseline prompt asked the broad question, "Is the rendered figure difficult or impossible to read?" I changed this to an explicit checklist of common readability issues (e.g., overlapping labels, tiny fonts). While it successfully caught the true positives for `hard_to_read` (TP=2), it caused the VLM to become hyper-sensitive, resulting in 10 False Positives and a category MCC of 0.0000.

Iteration 2: Hallucination Prevention (Dampening & Truncation)

To combat the hyper-sensitivity from Iteration 1, I introduced two techniques: 1. **Trajectory Truncation**: Sliced the trajectory to the final 2500 characters so the VLM is not confused by early agent mistakes. 2. **Benefit of the Doubt Clause**: Added a prompt constraint to only flag `hard_to_read` if the clutter is "catastrophic".

Quantitative Results (Self-Authored Runs): `execution_failure`: MCC=1.0000 (TP=3 TN=12 FP=0 FN=0)

`wrong_data`: MCC=0.0000 (TP=0 TN=7 FP=5 FN=0)

`wrong_chart`: MCC=0.0000 (TP=0 TN=10 FP=0 FN=2)

`hard_to_read`: MCC=0.2000 (TP=2 TN=2 FP=8 FN=0)

`macro_mcc`: 0.3000

Analysis: This was a massive success. The dampening clause successfully rescued 2 False Positives in `hard_to_read`, boosting the category MCC to 0.2000. Furthermore, trajectory truncation helped the VLM ignore bad data context, dropping `wrong_data` False Positives from 6 to 5. The overall `macro_mcc` reached 0.3000.

Iteration 3: Advanced Deterministic Execution Checks

Finally, to protect the VLM from evaluating broken runs on the hidden test set, I added pure Python deterministic checks before the VLM is queried: 1. **Traceback Regex**: Instantly fails runs containing "Traceback" or "SyntaxError" in the trajectory. 2. **Missing Import AST Check**: Fails runs that never import `matplotlib`, `seaborn`, or `plotly`. 3. **Blank Canvas Check**: Fails `figure.png` files under 2000 bytes or files comprised of a single solid color using `PIL.Image.getextrema()`.

Quantitative Results (Self-Authored Runs): `execution_failure`: MCC=1.0000 `wrong_data`: MCC=0.0000 `wrong_chart`: MCC=0.0000 `hard_to_read`: MCC=0.2000 `macro_mcc`: 0.3000

Analysis: The scores remained identical at 0.3000, meaning these strict deterministic constraints did not introduce any regressions or False Negatives. Instead, they provide a massive structural benefit: they shield the expensive VLM from being forced to hallucinate over completely broken or blank runs, creating a highly robust hybrid evaluation pipeline for the private test set.